Lista 1

Amelia Olejniczak

2024-10-19

Contents

1 Zadanie 1. **2**

2 Zadanie 2. **3**

2.1 Zadanie dodatkowe 3

2.2 Zadanie główne 3

3 Zadanie 3. **6**

4 Zadanie 4. **8**

4.1 Zadanie główne 8

4.2 Zadanie dodatkowe 11

1 Zadanie 1.

Aby wygenerować 1000 wektorów losowych z rozkładu dwuwymiarowego normalnego $N(0, I_{2\times 2})$ powtórzę tysiąckrotnie podwójne losowanie z rozkładu normalnego $N(0,1)$ za pomocą funkcji `replicate(1000, rnorm(2))`. Następnie za pomocą funkcji `plot()` wykonam wykres punktowy wylosowanych przeze mnie wartości. Dorysuję również osie przechodzące przez punkt (0,0), będący punktem opisującym wartość oczekiwaną tego rozkładu.

```
set.seed(12)
wektory1 <- replicate(1000, rnorm(2))
plot(wektory1[1,], wektory1[2,], asp=1, col="skyblue", xlab = " ", ylab = " ")
abline(h = 0, v = 0, col="deepskyblue4")
```

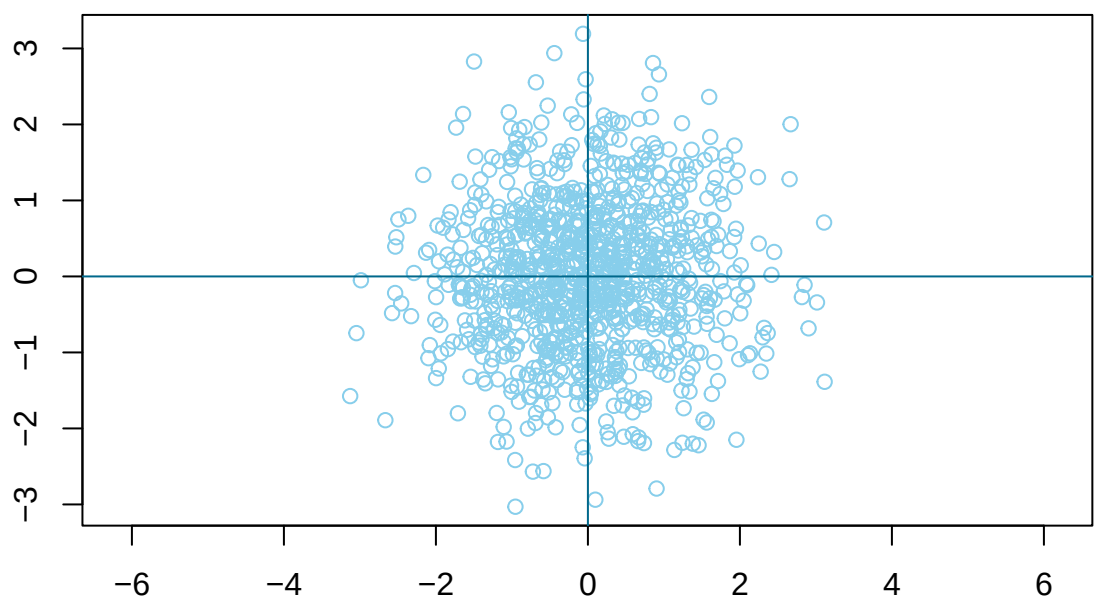


Figure 1: Wykres wektorów losowych rozkładu dwuwymiarowego normalnego $N(0,1)$

Przede wszystkim z tego wykresu można zauważyć, że punkty kumulują się w okolicy punktu (0, 0), a zatem parametru μ rozkładu normalnego. Im dalej od środka, tym bardziej się rozrzedzają, zatem większość wartości wynoszą około wartości oczekiwanej, co jest spodziewanym wynikiem. Punkty rozkładają się dość regularnie we wszystkie strony, a osie przechodzące przez punkt (0, 0) są również osiami symetrii tego rozkładu.

2 Zadanie 2.

2.1 Zadanie dodatkowe

Aby narysować osie symetrii, korzystam z wektorów własnych macierzy Σ, a następnie przechodzę z postaci parametrycznej prostej do postaci $y = ax + b$. Postać parametryczna wygląda w następujący sposób:

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} t + \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$$

gdzie $P_0 = (x_0, y_0)$ to punkt, przez który przechodzi prosta (w przypadku tego zadania będzie to punkt μ), a $V = (v_1, v_2)$ to wektor wyznaczający kierunek prostej. Po przekształceniu układu równań i wyliczeniu z niego a oraz b potrzebnych do wektora kierunkowego wychodzi, że

$$a = \frac{v_2}{v_1}, \quad b = y_0 - \frac{x_0 v_2}{v_1}$$

W R w funkcji `abline()` współczynniki a oraz b są zamienione miejscami (zatem prosta ma postać $y = bx - a$). Stworzyłam funkcję wyliczającą osie symetrii dla konkretnej wartości macierzy kowariancji Σ. Działa to oczywiście jedynie dla przypadku dwuwymiarowego z jakim mamy do czynienia w zadaniu drugim oraz trzecim.

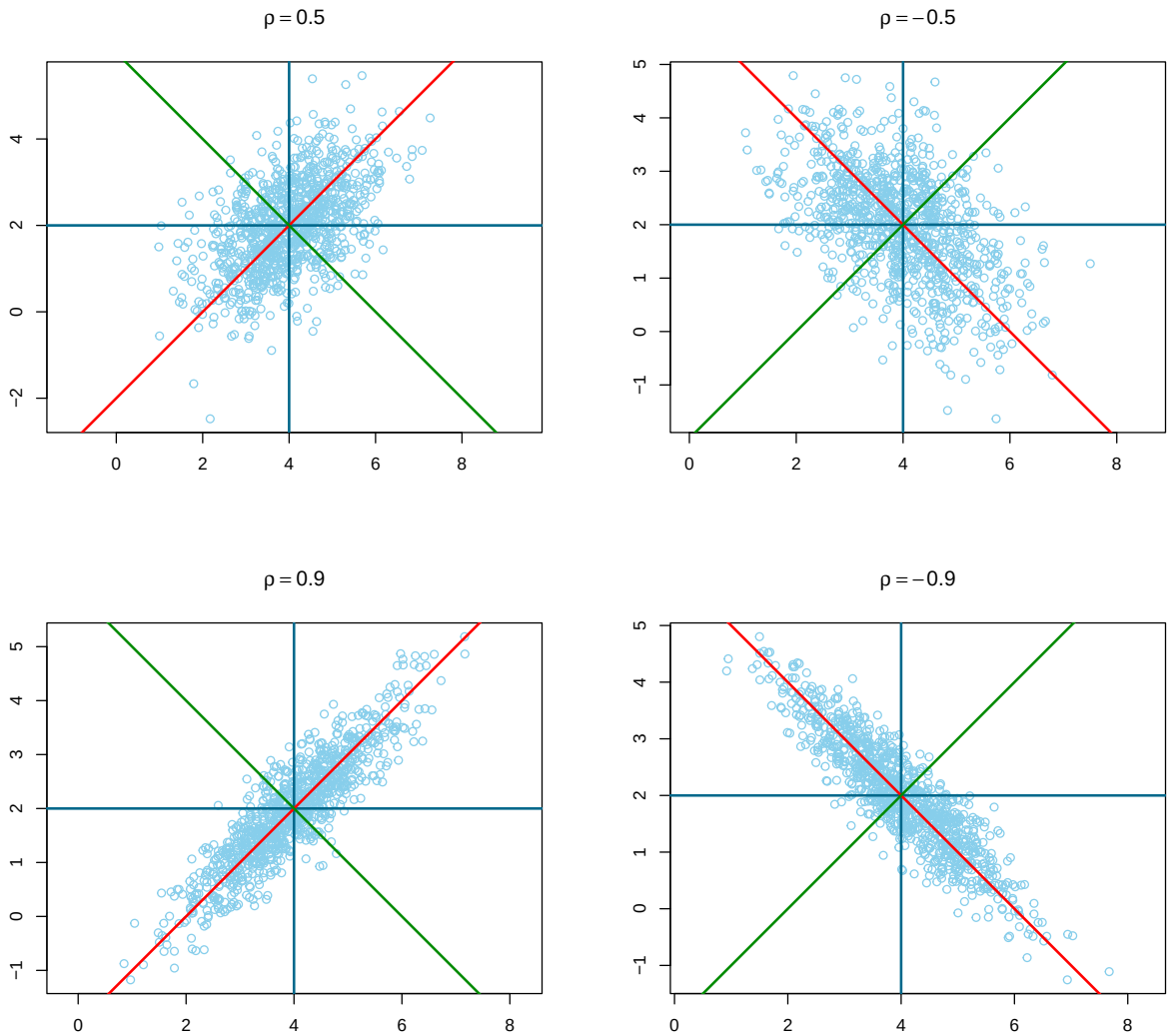
```
osie_symetrii <- function(sigma) {
  wektory_własne <- eigen(sigma)$vectors
  mi = c(4, 2)
  abline(a=mi[2]-((mi[1]*wektory_własne[2, 1])/wektory_własne[1, 1]),
         b=wektory_własne[2, 1]/wektory_własne[1, 1], lwd = 2, col="red")
  abline(a=mi[2]-((mi[1]*wektory_własne[2, 2])/wektory_własne[1, 2]),
         b=wektory_własne[2, 2]/wektory_własne[1, 2], lwd = 2, col="green4")
}
```

2.2 Zadanie główne

W celu zachowania przejrzystości i maksymalnego wykorzystania przestrzeni używam funkcji `par()`, która dla wartości `mfrow=c(2, 2)` ustawia wykresy w macierzy 2×2, dzięki czemu wszystkie będą widoczne na mniejszej przestrzeni. Następnie zapisuję wartość macierzy kowariancji Σ dla każdego z podpunktów i wyliczam macierz A z rozkładu Choleskyego za pomocą funkcji `chol()`, którą transmutuję, ze względu na to, że funkcja `chol()` wyrzuca nam taką macierz L, że $\Sigma = L^T L$, a my potrzebujemy takiej macierzy A, że $\Sigma = AA^T$. Wykonuję losowanie w taki sam sposób jak w zadaniu 1., otrzymując próbę losową X, a następnie wyliczam próbę losową Y ze wzoru $Y = AX + B$, gdzie A jest wyliczoną przez nas macierzą za pomocą rozkładu Choleskyego macierzy Σ, a B jest to wektor μ z naszego zadania. Następnie rysuję wykres za pomocą funkcji `plot()` oraz dorysowuję osie symetrii używając stworzonej przeze mnie wyżej funkcji. Tworzę także osie $x = 2$ oraz $y = 4$, aby widoczne było, w którym miejscu jest punkt odpowiadający za parametr $\mu = (4, 2)$

```
set.seed(27)
par(mfrow=c(2,2))

sigma2a <- matrix(c(1, 0.5, 0.5, 1), nrow = 2, ncol = 2)
A2a <- t(chol(sigma2a))
wektory <- replicate(1000, rnorm(2))
wektory2a <- A2a %*% wektory + c(4, 2)
plot(wektory2a[1,], wektory2a[2,], asp=1, col="skyblue", xlab = " ", ylab = " ",
```

```r
     main = bquote(rho == 0.5))
abline(h = 2, v = 4, col="deepskyblue4", lwd = 2)
osie_symetrii(sigma2a)

sigma2b <- matrix(c(1, -0.5, -0.5, 1), nrow = 2, ncol = 2)
A2b <- t(chol(sigma2b))
wektory <- replicate(1000, rnorm(2))
wektory2b <- A2b %*% wektory + c(4, 2)
plot(wektory2b[1,], wektory2b[2,], asp=1, col="skyblue", xlab = " ", ylab = " ",
     main = bquote(rho == - 0.5))
abline(h = 2, v = 4, col="deepskyblue4", lwd = 2)
osie_symetrii(sigma2b)

sigma2c <- matrix(c(1, 0.9, 0.9, 1), nrow = 2, ncol = 2)
A2c <- t(chol(sigma2c))
wektory <- replicate(1000, rnorm(2))
wektory2c <- A2c %*% wektory + c(4, 2)
plot(wektory2c[1,], wektory2c[2,], asp=1, col="skyblue", xlab = " ", ylab = " ",
     main = bquote(rho == 0.9))
abline(h = 2, v = 4, col="deepskyblue4", lwd = 2)
osie_symetrii(sigma2c)

sigma2d <- matrix(c(1, -0.9, -0.9, 1), nrow = 2, ncol = 2)
A2d <- t(chol(sigma2d))
wektory <- replicate(1000, rnorm(2))
wektory2d <- A2d %*% wektory + c(4, 2)
plot(wektory2d[1,], wektory2d[2,], asp=1, col="skyblue", xlab = " ", ylab = " ",
     main = bquote(rho == - 0.9))
abline(h = 2, v = 4, col="deepskyblue4", lwd = 2)
osie_symetrii(sigma2d)
```

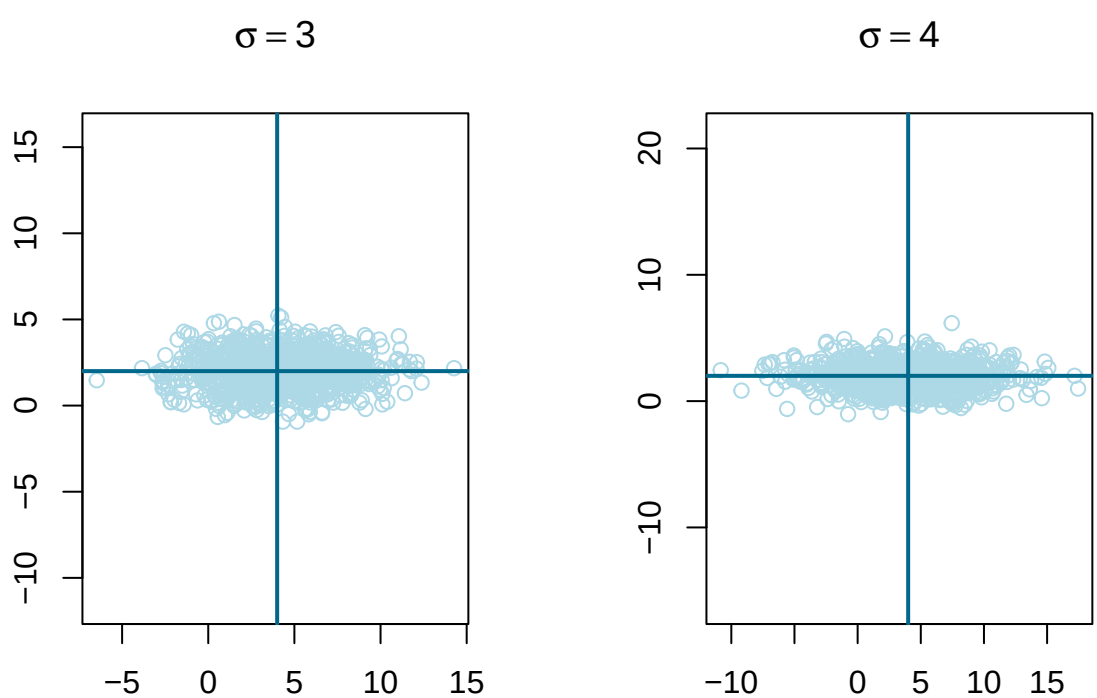


Przede wszystkim, na wyznaczonych wykresach widać, że osie symetrii zostały wyznaczone prawidłowo. Przyglądając się parametrowi ρ można zauważać, że dla większej wartości tego parametru, wykres jest bardziej rozciągnięty i wydaje się przez to bardziej płaski. Poza tym, "chmury" dla wartości, które różnią się jedynie swoim znakiem są swoimi odbiciami wzdłuż osi $y = 4$, która również jest zaznaczona na wykresie. Punkty kumulują się wzdłuż czerwonej osi symetrii, w szczególności wokół punktu (4, 2), będącego wartością oczekiwaną rozkładu.

3 Zadanie 3.

Zadanie wykonuję w ten sam sposób co Zadanie 2., z tą różnicą, że tym razem w funkcji `par()` wpisuję `mfrow=c(1, 2)`, ponieważ potrzebuję wyrysować jedynie dwa wykresy. Ponownie dodaję do nich również te same proste $x = 2$ oraz $y = 4$, ponieważ parametr $\mu = (4, 2)$ jest taki sam.

```
par(mfrow=c(1,2))
set.seed(72)

sigma3a <- matrix(c(9, 0, 0, 1), nrow = 2, ncol = 2)
A3a <- t(chol(sigma3a))
wektory <- replicate(1000, rnorm(2))
wektory3a <- A3a %*% wektory + c(4, 2)
plot(wektory3a[1,], wektory3a[2,], asp=1, col="lightblue", xlab = " ", ylab = " ",
     main = bquote(sigma == 3))
abline(h = 2, v = 4, col="deepskyblue4", lwd = 2)

sigma3b <- matrix(c(16, 0, 0, 1), nrow = 2, ncol = 2)
A3b <- t(chol(sigma3b))
wektory <- replicate(1000, rnorm(2))
wektory3b <- A3b %*% wektory + c(4, 2)
plot(wektory3b[1,], wektory3b[2,], asp=1, col="lightblue", xlab = " ", ylab = " ",
     main = bquote(sigma == 4))
abline(h = 2, v = 4, col="deepskyblue4", lwd = 2)
```



Jak widać, podobnie jak w poprzednim podpunkcie, dla większej wartości parametru σ otrzymujemy bardziej

spłaszczony i rozciągnięty wykres. Tym razem, są one jednak wzdłuż osi $y = 4$, więc ta oś jest również osią symetrii tego wykresu. Może tak być ze względu na to, że macierz kowariancji Σ jest w tym przypadku macierzą diagonalną. Wynikałoby wtedy z tego, że liczby znajdujące się poza przekątną w macierzy 2×2 odpowiadają za obrót punktów znajdujących się na wykresie. Ponownie, większość punktów znajduje się w okolicy wartości oczekiwanej $\mu = (4, 2)$.

4 Zadanie 4.

4.1 Zadanie główne

Aby wykonać podpunkt (a) zadania 4., losuję elementy poszukiwanego wektora losowego za pomocą funkcji `rnorm()` dla próby rozmiaru 100000, ponieważ wtedy, za pomocą funkcji `matrix(wektory, nrow = 1000, ncol = 100)` jestem w stanie utworzyć z takiej ilości liczb losowych wektor 1000×100. Ma on ustalone 1000 wierszy, zatem w nich jest zapisane 1000 wektorów losowych.

Aby wyliczyć macierz A taką, jak w poleceniu (b) z zadania 4. korzystam z rozkładu Choleskyego. Tym razem zmienną $A4$ wyliczam za pomocą funkcji `chol()` nie wykonując transpozycji macierzy, ponieważ

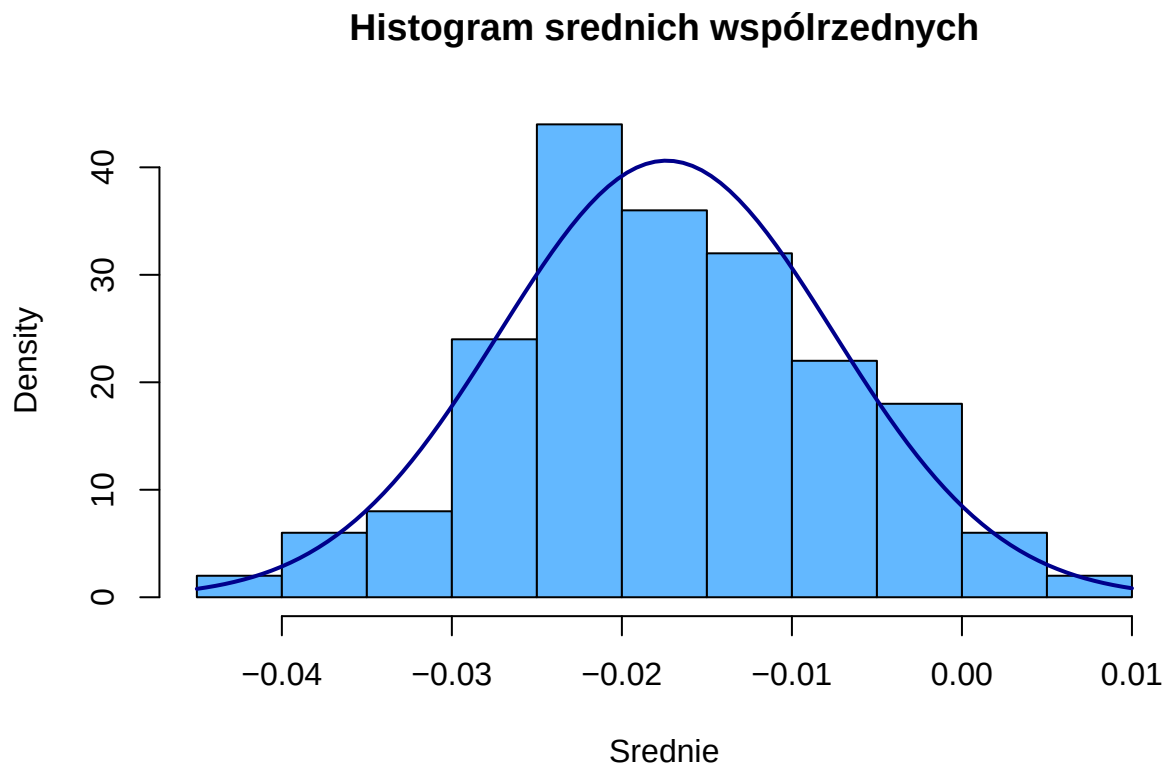
$$XA = (X^T)^T \cdot (A^T)^T = (A^T X^T)^T$$

W poprzednich zadaniach kolejne wektory losowe były zapisane w kolumnach, zatem w tym przypadku, ponieważ kolejne wektory są zapisywane w wierszach, należy transponować macierz X aby móc skorzystać z rozkładu Choleskyego. A^T będzie więc wtedy macierzą wyliczoną z rozkładu Choleskyego macierzy Σ opisanej w zadaniu dla wektorów losowych zapisanych w macierzy X^T. Z tego względu macierz A będzie macierzą z rozkładu Choleskyego macierzy Σ.

```r
set.seed(6)

wektory <- rnorm(1000 * 100)
wektory4 <- matrix(wektory, nrow = 1000, ncol = 100)

library(matlib)
sigma4 <- matrix(0.9, 100, 100)
diag(sigma4) <- 1
A4 <- chol(sigma4)
x <- wektory4 %*% A4
```

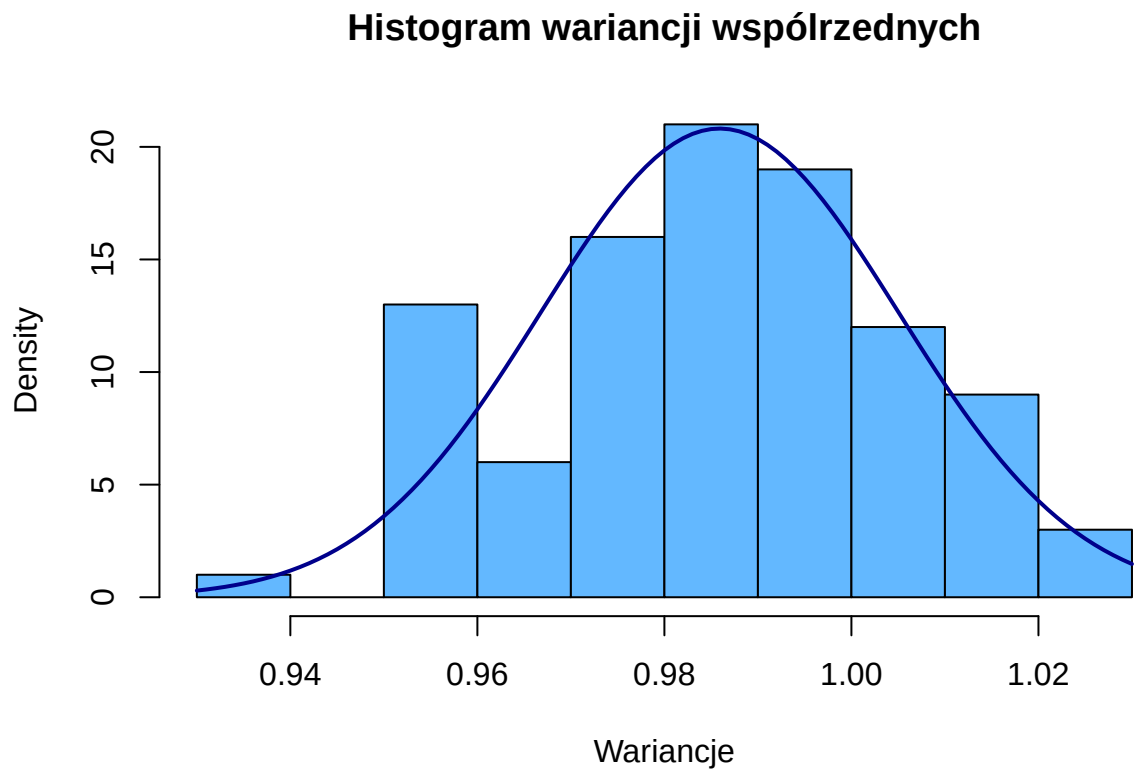
W ostatnim podpunkcie wyrysowuję histogramy za pomocą funkcji `hist()`. Najważniejsze tutaj jest w przypadku kowariancji wzięcie jedynie wartości pod przekątną, gdyż nad przekątną znajdują się te same wartości, natomiast na przekątnej znajdują się wariancje. Do każdego z histogramów dodałam również krzywe, określające rozkład normalny dla podanych zmiennych, aby zobaczyć, jak blisko jest histogram.

```r
srednie <- colMeans(x)
hist(srednie, col = "steelblue1", freq = FALSE, main = "Histogram średnich współrzędnych",
     xlab = "Średnie")
curve(dnorm(x, mean = mean(srednie), sd = sd(srednie)), col = "darkblue", lwd = 2, add = TRUE)
```



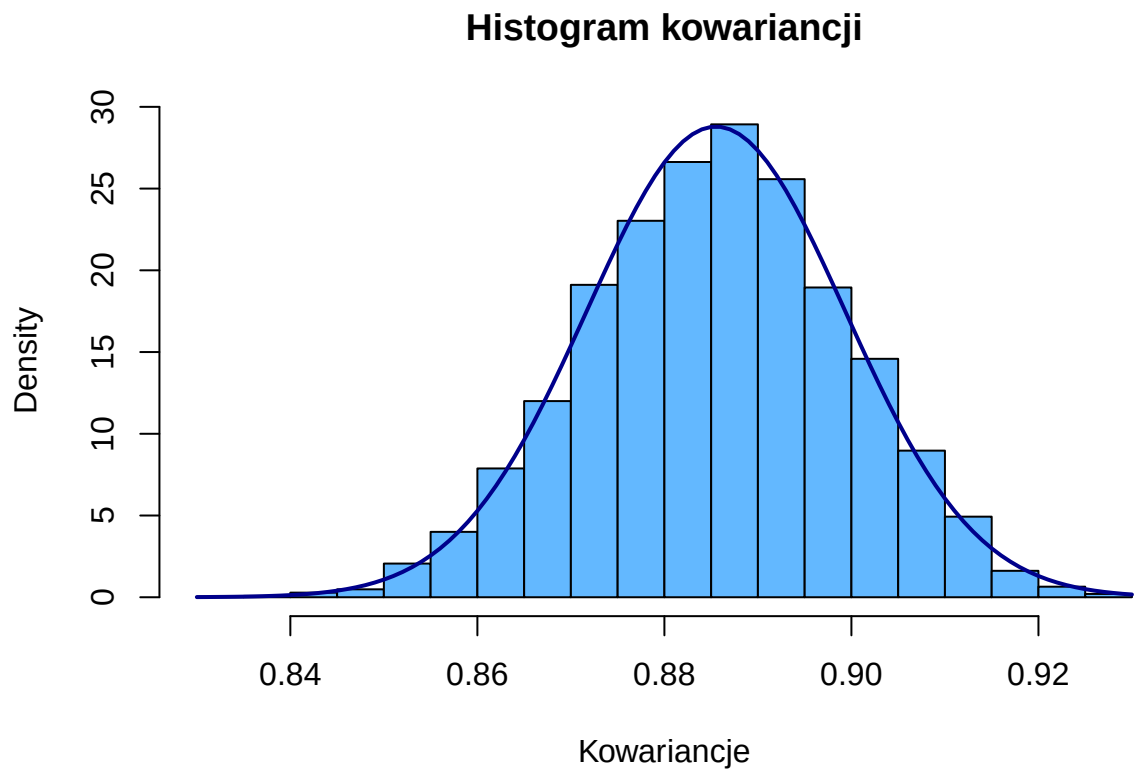
Po powyższym histogramie powinniśmy oczekiwać, że wartości będą się znajdować gdzieś w okolicy punktu 0 i jak można zauważyć, tak właśnie jest. Histogram jest delikatnie przesunięty, tak, że najwyższy słupek jest dla wartości od około -0.035 do -0.02, natomiast jest to niewiele dalej od zera. Patrząc na linię wyznaczającą gęstość rozkładu normalnego można zauważyć, że w porządku przybliża ona rozkład tych średnich.

```
wariancje <- apply(x, 2, var)
hist(wariancje, col = "steelblue1", freq = FALSE, main = "Histogram wariancji współrzędnych",
     xlab = "Wariancje")
curve(dnorm(x, mean = mean(wariancje), sd = sd(wariancje)), col = "darkblue", lwd = 2, add = TRUE)
```



W przypadku histogramu dla wariancji, dla którego spodziewamy się, że większość obserwacji będzie "wpadać" w słupek w okolicach wartości 1, również można zauważyć lekkie przesunięcie. W tym przypadku najwyższy słupek jest to ten dla wartości między 0.98, a 0.99. Różnica od wartości 1 nie jest jednak na tyle znacząca, żeby się tym przejmować i można przyjąć, że wariancje są wystarczająco bliskie wartości 1. Krzywa opisująca gęstość rozkładu normalnego jest w tym przypadku również całkiem niezłym przybliżeniem, chociaż w tym przypadku nieco gorszym niż dla histogramu średnich.

```
macierz_kow <- cov(x)
kowariancje <- macierz_kow[lower.tri(macierz_kow)]
hist(kowariancje, col = "steelblue1", freq = FALSE, main = "Histogram kowariancji",
     xlab = "Kowariancje")
curve(dnorm(x, mean = mean(kowariancje), sd = sd(kowariancje)), col = "darkblue", lwd = 2, add = TRUE)
```



Dla kowariancji wartości ponownie znajdują się w pobliżu oczekiwanej wartości 0.9, chociaż ponownie nie bez delikatnegop przesunięcia. Tym razem wartości są najczęsciej w słupkach między 0.88, a 0.89, co, ponownie, nie jest daleko na osi od 0.9. Co warto zauważyć, to że histogram kowariancji praktycznie idealnie nakłada się na prostą opisującą gęstość rozkładu normalnego. Na pewno jest to lepsze przybliżenie dla kowariancji niż pozostałych parametrów, czyli wariancji oraz średniej.

4.2 Zadanie dodatkowe

Pierwszym działaniem w funkcji, która ma wyliczać rozkład Choleskyego dla macierzy Σ będzie wzięcie liczby wierszy macierzy Σ. Ponieważ jest to macierz kwadratowa, nie ma znaczenia, czy użyję tutaj `nrow()` czy `ncol()`, ponieważ obie liczby są takie same. Następnie tworzę macierz, którą oznaczam jako `c` i jest to macierz o takich samych wymiarach, jak Σ, jednak na każdym miejscu tej macierzy znajduje się zero. Następnie liczę potrzebną do rozkładu Choleskyego sumę. W tej pętli `i` oznacza numer wiersza, natomiast `j` numer kolumny. Idziemy po kolei wierszami i wypełniamy jedynie te kolumny, których numer jest mniejszy lub równy numerowi wiersza. Dzięki temu wypełniamy jedynie dolny trójkąt, a w górnym na pewno zostaną zera. Następnie korzystam z dwóch różnych wzorów w zależności od tego, czy `i = j` czy nie. Na koniec funkcja "wyrzuca" gotową macierz dolnotrójkątną.

```
cholesky <- function(sigma) {
  n <- nrow(sigma)
  c <- matrix(0, n, n)

  for (i in 1:n) {
    for (j in 1:i) {
      sum_k <- sum(c[i, 1:(j-1)] * c[j, 1:(j-1)])
```

```r
      if (i == j) {
        c[i, j] <- sqrt(sigma[i, i] - sum_k)
      } else {
        c[i, j] <- (sigma[i, j] - sum_k) / c[j, j]
      }
    }
  }
  return(c)
}
```

Aby pokazać, że funkcja działa, wywołam dwie macierze. Jedną wyliczoną za pomocą transpozycji funkcji `chol()` z R, a drugą wyliczoną za pomocą mojej funkcji `cholesky()` dla macierzy `sigma2b`, a zatem tej z podpunktu (b) zadania drugiego. Można zauważyć, że są one takie same.

```r
cholesky(sigma2b)
```

```
##      [,1]      [,2]
## [1,]  1.0 0.0000000
## [2,] -0.5 0.8660254
```

```r
t(chol(sigma2b))
```

```
##      [,1]      [,2]
## [1,]  1.0 0.0000000
## [2,] -0.5 0.8660254
```